

GUIDE COMPLET DE SOUTENANCE

Industrial Equipment Health Platform

Scripts individuels, explication du projet, démonstration en direct et réponses aux questions

MLOps et DataOps

Équipe de 7 présentateurs

Version finale préparée pour la soutenance. Les valeurs de production doivent être vérifiées le jour de la démonstration.

1. Comment utiliser ce guide

Un seul document pour préparer les prises de parole et la démonstration.

- Chaque personne apprend les idées et les transitions, pas chaque phrase mot à mot.
- Chaque intervention reste sous cinq minutes, conformément aux consignes de soutenance.
- Une seule personne pilote l'écran pendant la démonstration afin d'éviter les changements de session.
- Les valeurs dynamiques, notamment le commit, le run GitHub Actions et la version déployée, sont contrôlées juste avant la soutenance.
- Si un service ne répond pas, utiliser les captures datées et les réponses JSON conservées hors ligne.

Message central

Le projet ne se limite pas à un modèle. Il relie données, qualité, entraînement, registre, API, conteneurs, déploiement et monitoring dans une chaîne reproductible.

2. Le projet expliqué simplement

Cette partie permet de comprendre l'ensemble avant d'apprendre les scripts.

Le problème

Une panne industrielle peut provoquer un arrêt coûteux. La maintenance corrective intervient après la panne. La maintenance préventive intervient selon un calendrier. La maintenance prédictive utilise les mesures réelles de l'équipement pour estimer le moment utile d'intervention.

La réponse du projet

La plateforme estime la Remaining Useful Life, appelée RUL, c'est à dire le nombre de cycles restant avant une défaillance. Une API renvoie cette estimation avec un niveau de risque, la version du modèle et la latence.

Les données

Le jeu NASA C-MAPSS simule la dégradation de turboréacteurs. FD001 à FD004 couvrent 709 moteurs d'entraînement, 707 moteurs de test, trois réglages opérationnels et 21 capteurs. L'évaluation finale utilise le dernier cycle observé de chaque moteur de test.

Le résultat scientifique

Élément	Valeur à présenter
Modèle retenu	HistGradientBoostingRegressor
MAE	12,56 cycles
RMSE	16,93 cycles
Score NASA	4 466
Population finale	707 moteurs de test
Champion MLflow observé	Version 2

Interprétation

La MAE mesure l'erreur typique. La RMSE accentue les grandes erreurs. Le score NASA pénalise davantage une surestimation de la RUL, car elle peut retarder une maintenance nécessaire.

3. Architecture complète

Le trajet d'une donnée depuis la NASA jusqu'à la démonstration.

Étape	Technologie	Rôle
Source	NASA C-MAPSS	Trajectoires de moteurs jusqu'à la défaillance
Ingestion	dlt	Chargement déclaratif et schéma
Stockage	DuckDB	Tables brutes, intermédiaires et marts
Transformation	dbt	SQL, tests et documentation
Orchestration	Dagster	Dépendances, matérialisation et planning quotidien
Modélisation	scikit-learn	Features temporelles et comparaison de cinq modèles
Expériences	MLflow	Runs, métriques, artefacts et registre
Service	FastAPI	Validation, santé, prédiction et monitoring
Conteneurs	Docker Compose	API, MLflow, Dagster et volumes persistants
Déploiement	Komodo	Stack, conteneurs, logs et cycle de livraison
Validation	GitHub Actions	Tests, build et smoke test

Automatisation réelle

Dagster lance le pipeline complet tous les jours à 06:00, heure de Casablanca. Le rapport de drift est généré pour le monitoring. Il ne déclenche pas le réentraînement. Après l'entraînement, un candidat MLflow devient champion uniquement si sa MAE et sa RMSE ne se dégradent pas.

Déploiement présenté

Komodo gère la Stack sur le serveur partagé de l'université. L'API FastAPI est exposée sur le port 3402, MLflow sur 3401 et Dagster sur 3403. L'interface publique existante communique avec cette API. Pendant la soutenance, il faut distinguer clairement l'interface, qui affiche le produit, et les services MLOps, qui tournent sur le serveur.

4. Liens à préparer

Ouvrir les pages avant le début de la soutenance et les garder dans cet ordre.

[Application publique](https://industrial-equipment-health-platfor.vercel.app/)

<https://industrial-equipment-health-platfor.vercel.app/>

[Komodo](https://komodo.s3.fsbm.ma/)

<https://komodo.s3.fsbm.ma/>

[FastAPI](http://exp.s3.fsbm.ma:3402/)

<http://exp.s3.fsbm.ma:3402/>

[Documentation FastAPI](http://exp.s3.fsbm.ma:3402/docs)

<http://exp.s3.fsbm.ma:3402/docs>

[Santé FastAPI](http://exp.s3.fsbm.ma:3402/health)

<http://exp.s3.fsbm.ma:3402/health>

[MLflow](http://exp.s3.fsbm.ma:3401/)

<http://exp.s3.fsbm.ma:3401/>

[Dagster](http://exp.s3.fsbm.ma:3403/)

<http://exp.s3.fsbm.ma:3403/>

[Dépôt GitHub](https://github.com/Maaskk/industrial-equipment-health-platform)

<https://github.com/Maaskk/industrial-equipment-health-platform>

[GitHub Actions](https://github.com/Maaskk/industrial-equipment-health-platform/actions)

<https://github.com/Maaskk/industrial-equipment-health-platform/actions>

[Pull request équipe et scripts](https://github.com/Maaskk/industrial-equipment-health-platform/pull/12)

<https://github.com/Maaskk/industrial-equipment-health-platform/pull/12>

Attention

Les ports sont ceux de la plage 34XX du groupe. Si le professeur ou l'administrateur modifie le routage, vérifier les trois liens serveur avant la soutenance.

5. Répartition et chronométrage

La présentation orale dure 29 minutes 30, puis la démonstration dure 5 minutes.

Ordre	Personne	Durée	Responsabilité présentée
1	Ilyass	4 min 15	Contexte, données, EDA et KPI
2	Mohamed	4 min	Architecture DataOps et Dagster
3	Hamza	4 min	Qualité, lignage et CI
4	Mouhcine	4 min 15	Features, modèles et métriques
5	Ossama	4 min 30	MLflow, réentraînement, Docker et Komodo
6	Hajar	4 min	FastAPI et parcours de démonstration
7	Aya Moujoud	4 min 30	QA, monitoring, Komodo et limites
Équipe	Démonstration	5 min	Preuves en direct

Durée totale

34 minutes 30. La limite correspondant à sept étudiants est 35 minutes.

6. Scripts individuels

Texte complet de chaque présentateur avec décision technique et questions probables.

Ilyass

Diapositives 1 à 3. Durée cible : 4 minutes 15.

Texte à présenter

« Bonjour. Nous allons présenter Industrial Equipment Health Platform, notre plateforme de maintenance prédictive. L'objectif du projet n'est pas seulement d'entraîner un modèle. Nous avons construit une chaîne complète qui part des données, contrôle leur qualité, entraîne et versionne le modèle, puis expose la prédiction dans une application déployée. Chaque membre va expliquer la partie qu'il a prise en charge. »

« Dans l'industrie, on distingue trois stratégies. La maintenance corrective intervient après la panne. Elle est simple, mais elle arrive trop tard et peut provoquer un arrêt coûteux. La maintenance préventive intervient selon un calendrier. Elle réduit le risque, mais elle peut remplacer un composant encore utilisable. La maintenance prédictive utilise l'état réel de l'équipement pour choisir le bon moment. Notre question est donc : combien de cycles reste-t-il avant la défaillance ? Cette estimation est la RUL, la Remaining Useful Life. »

« Ma contribution porte sur l'analyse des données, l'EDA et les indicateurs métier. Nous utilisons NASA C-MAPSS, un jeu qui simule la dégradation de moteurs de turboréacteurs. Les quatre sous-ensembles FD001 à FD004 contiennent 160 359 lignes d'entraînement et 104 897 lignes de test. Nous avons 709 moteurs pour l'entraînement, 707 pour le test, trois réglages opérationnels et 21 capteurs. »

« L'analyse exploratoire montre que les durées de vie sont dispersées, que certains capteurs sont presque constants et que d'autres présentent des tendances utiles. J'ai conservé l'ordre des cycles afin de ne jamais utiliser une information future. Les indicateurs du produit sont la RUL, le niveau de risque, la version du modèle et la latence. Les seuils sont simples : risque élevé sous 30 cycles, moyen entre 30 et 80, faible au-dessus de 80. »

« Ces données constituent l'entrée du pipeline. Mohamed va maintenant expliquer comment elles sont ingérées, stockées et orchestrées. »

Décision technique à défendre

La séparation temporelle et la séparation par moteur évitent les fuites de données. La RUL est plafonnée à 125 cycles afin de limiter l'effet de la longue phase initiale où la dégradation est peu visible.

Questions probables

- Pourquoi C-MAPSS ? Parce qu'il fournit des trajectoires multivariées jusqu'à la défaillance avec une vérité terrain RUL adaptée à un projet reproductible.
- Pourquoi trois niveaux de risque ? Ils transforment une valeur continue en priorité opérationnelle compréhensible. Ce ne sont pas des classes apprises.
- Quelle limite principale ? Les données sont simulées et ne remplacent pas une télémétrie industrielle réelle.

Mohamed

Diapositives 4 et 5. Durée cible : 4 minutes.

Texte à présenter

« Ma contribution concerne la partie DataOps et l'orchestration. L'architecture commence avec les fichiers NASA C-MAPSS. dlt les charge de façon déclarative. DuckDB conserve les tables brutes, les tables intermédiaires et les marts. dbt transforme les données et exécute les tests. Le mart final alimente les features du modèle. Ensuite, MLflow conserve les expériences, FastAPI sert le champion, Docker décrit les services et Komodo gère la Stack sur le serveur partagé. »

« J'ai choisi DuckDB parce que le volume, environ 265 000 lignes brutes, tient sur une seule machine. Il n'est donc pas nécessaire d'ajouter un serveur de base de données. Ce choix simplifie l'installation, la CI et la démonstration, tout en gardant SQL, les tables et les requêtes reproductibles. »

« Dagster impose l'ordre du pipeline. L'ingestion doit réussir avant dbt. Les tests dbt doivent réussir avant la création des features. Le training produit ensuite un candidat MLflow. Le rapport de drift est généré après le training. Cette dépendance empêche une étape de contourner un contrôle précédent. »

« Le job final est planifié chaque jour à 06:00 dans le fuseau de Casablanca. Il s'agit bien d'un réentraînement automatique planifié. Le rapport de drift ne déclenche pas ce job. Il sert seulement au monitoring et à l'analyse. Cette distinction évite de présenter une boucle fermée qui n'existe pas dans le code. »

« Le même point d'entrée peut être utilisé en local, dans GitHub Actions et dans Docker. Hamza va maintenant montrer comment les contrats et les tests empêchent des données invalides d'atteindre le modèle. »

Décision technique à défendre

DuckDB a été préféré à une base distribuée parce qu'il couvre le volume réel du projet avec moins d'administration. Dagster a été choisi pour rendre les dépendances visibles, jouables et contrôlables.

Questions probables

- Pourquoi dlt et dbt ensemble ? dlt gère l'ingestion et le schéma de chargement.

dbt gère les transformations SQL, les tests et la documentation.

- Que se passe-t-il si dbt échoue ? Les assets suivants ne s'exécutent pas.
- Le drift lance-t-il un entraînement ? Non. Le job quotidien lance le pipeline.

Hamza

Diapositives 6 et 7. Durée cible : 4 minutes.

Texte à présenter

« Ma contribution porte sur la qualité des données, le lignage, les tests et la validation continue. Les contrats définissent les colonnes attendues, leurs types et les règles principales. Les tests contrôlent les valeurs nulles, les domaines, l'unicité du couple moteur-cycle et la cohérence des volumes. Une erreur n'est pas simplement enregistrée : elle bloque la suite du pipeline. »

« Le lignage permet de répondre à une question simple : d'où vient une prédiction ? Nous pouvons repartir du journal de prédiction, retrouver la version du modèle dans MLflow, le mart de features, les tables dlt et les fichiers NASA d'origine. Cette traçabilité est nécessaire pour analyser une erreur et pour reproduire un résultat. »

« J'ai également travaillé sur les contrôles dans GitHub Actions. La CI vérifie les données et dbt, les tests Python, la compilation et les règles statiques. Elle exécute ensuite l'entraînement et le registre, construit l'image Docker, puis effectue un smoke test sur la santé et la prédiction. Pendant la démonstration, j'ouvre la dernière exécution verte et j'annonce l'identifiant et le commit visibles à l'écran. »

« Le rôle de ma partie dans le pipeline est donc de créer une barrière avant le modèle et avant le déploiement. Un code qui fonctionne mais utilise des données incorrectes ne doit pas être livré. Une fois ces contrôles validés, Mouhcine peut comparer les modèles avec un protocole fiable. »

Décision technique à défendre

Les contrôles sont bloquants et exécutés à plusieurs niveaux : contrats, tests dbt, tests Python et smoke test Docker. Cette redondance protège des erreurs différentes au lieu de dépendre d'un seul test final.

Questions probables

- Différence entre contrat et test ? Le contrat décrit la structure attendue.

Le test vérifie une propriété sur les données réellement chargées.

- Pourquoi un smoke test après le build ? Il vérifie que l'image démarre et que

l'API répond, pas seulement que le code compile.

- Que prouve la CI ? Elle prouve la reproductibilité des contrôles sur le commit

testé, pas la qualité future de toutes les données industrielles.

Mouhcine

Diapositives 8 et 9. Durée cible : 4 minutes 15.

Texte à présenter

« Ma contribution concerne le feature engineering, la comparaison des modèles et l'évaluation finale. Les mesures brutes ne décrivent qu'un cycle isolé. Pour capturer la dégradation récente, nous calculons sur une fenêtre de cinq cycles la moyenne, l'écart type et la pente de chaque capteur. Avec le cycle, le sous-ensemble, les réglages opérationnels et les mesures, le modèle utilise 89 variables internes. »

« Nous avons comparé cinq familles dans le même protocole : une baseline moyenne, un Ridge utilisant surtout le cycle, un Ridge sur les variables brutes, une forêt aléatoire et un HistGradientBoostingRegressor. La séparation est faite par moteur afin qu'un moteur de validation ne soit jamais présent dans l'entraînement. »

« L'évaluation standard utilise le dernier cycle observé des 707 moteurs de test. La MAE indique l'erreur absolue moyenne en cycles. La RMSE donne plus de poids aux grandes erreurs. Le score NASA est asymétrique et pénalise davantage une surestimation de la RUL, car annoncer trop de durée de vie peut retarder une maintenance nécessaire. »

« Le modèle retenu est le HistGradientBoostingRegressor. Il obtient une MAE de 12,56 cycles, une RMSE de 16,93 cycles et un score NASA de 4 466. La forêt aléatoire obtient une MAE proche, 12,73 cycles. Le choix ne repose donc pas sur une différence spectaculaire de MAE. Le gradient boosting offre aussi le meilleur score NASA et une inférence adaptée au service. »

« Le modèle retenu devient un candidat dans MLflow. Ossama va expliquer comment nous suivons cette expérience, décidons d'une promotion et déployons le service. »

Décision technique à défendre

Le protocole final évalue un point par moteur au dernier cycle, ce qui correspond à la convention C-MAPSS. Les modèles sont comparés avec trois métriques, pas avec la MAE seule.

Questions probables

- Pourquoi une fenêtre de cinq cycles ? Elle capture une tendance récente sans trop lisser les changements.
- Pourquoi le score NASA ? Une surestimation est plus risquée qu'une sous-estimation dans un contexte de maintenance.
- Pourquoi ne pas choisir automatiquement la forêt aléatoire ? Sa MAE est proche, mais son score NASA est moins bon et son coût d'inférence est supérieur.

Ossama

Diapositives 10 à 12. Durée cible : 4 minutes 30.

Texte à présenter

« Ma contribution relie l'entraînement au déploiement. J'ai pris en charge l'intégration MLOps, MLflow, la règle de promotion, Docker et la configuration Komodo. Chaque entraînement crée un run MLflow avec les paramètres, les métriques et les artefacts. Le modèle est ensuite enregistré comme candidat dans le registre. L'API charge l'alias champion, pas un numéro codé en dur. Le champion observé en production est actuellement la version 2. »

« La promotion est contrôlée. Le premier candidat devient champion lorsqu'aucun alias n'existe. Pour les candidats suivants, le code compare la MAE et la RMSE avec le champion. Le nouvel alias n'est appliqué que si les deux métriques ne se dégradent pas. Si les métriques du champion manquent, la promotion est bloquée. La décision est conservée dans MLflow et dans un rapport JSON. »

« Le réentraînement automatique existe. Dagster lance le job complet tous les jours à 06:00, heure de Casablanca. Le drift ne déclenche pas ce job. Il est calculé après l'entraînement et sert au monitoring. Nous avons donc une automatisation planifiée et une promotion conditionnelle, mais pas une boucle de réentraînement déclenchée par la dérive. »

« Pour l'industrialisation, GitHub Actions valide le commit, construit l'image et réalise le smoke test. Juste avant la soutenance, nous vérifions la révision déployée et le dernier run vert au lieu de réciter une ancienne valeur. Docker Compose décrit FastAPI, MLflow et Dagster. Komodo gère cette Stack sur le serveur partagé. La production de démonstration utilise FD001 pour rester légère, alors que l'évaluation scientifique couvre FD001 à FD004 et 707 moteurs. »

« Hajar va maintenant présenter le contrat FastAPI utilisé par l'application et le parcours de démonstration. »

Décision technique à défendre

FastAPI charge `models:/industrial-equipment-health-model@champion`. Cette URI découple le service d'un numéro de version. La règle de promotion évite qu'un simple entraînement planifié remplace automatiquement un meilleur modèle.

Questions probables

- Pourquoi comparer MAE et RMSE ? La MAE mesure l'erreur typique et la RMSE protège contre l'augmentation des grandes erreurs.
- Que se passe-t-il si MLflow est indisponible ? Le service de production doit échouer clairement ; le repli local est réservé aux tests explicites.
- Pourquoi FD001 en production ? Pour limiter les ressources de la démo. Le modèle final reste évalué sur les quatre sous-ensembles.

Hajar

Diapositives 13 et 14. Durée cible : 4 minutes.

Texte à présenter

« Ma contribution concerne le contrat FastAPI et le parcours de démonstration. L'API constitue la frontière entre le modèle et l'application. La route GET /health confirme que le service fonctionne et indique la version du modèle. GET /demo-payload fournit un exemple valide. POST /predict exécute la prédiction. Les routes sous /api alimentent les vues de flotte, l'historique d'un moteur et le monitoring. »

« La validation des entrées est stricte avec Pydantic. La requête contient l'identifiant du moteur, le cycle, les réglages et les mesures de capteurs. Le payload de démonstration fournit 88 valeurs et le cycle complète le vecteur de 89 variables internes. Un champ inconnu ou une structure incorrecte provoque une erreur 422 au lieu d'être ignoré silencieusement. »

« La réponse contient la RUL prédite, le niveau de risque, la version du modèle et la latence. Cette structure permet d'afficher une décision compréhensible et de savoir quel modèle l'a produite. Le contrat est également utilisé par les tests, ce qui évite d'avoir une interface de démonstration différente du service réel. »

« Pendant la démo, je commence par la vue flotte pour montrer les priorités. Je sélectionne ensuite un moteur et son historique de cycles. Je lance une prédiction et je vérifie la réponse. Enfin, j'ouvre les preuves de plateforme. La visualisation du moteur sert à expliquer les mesures ; ce n'est pas une simulation physique validée. »

« Aya va maintenant présenter les contrôles de production, le monitoring, l'état de la Stack Komodo et les limites du projet. »

Décision technique à défendre

Le contrat est strict et versionnable. L'API renvoie les informations nécessaires à l'audit, notamment la version du modèle et la latence, au lieu de renvoyer une valeur RUL isolée.

Questions probables

- Pourquoi 88 valeurs reçues et 89 variables internes ? Le cycle complète le vecteur construit pour le modèle.
- Pourquoi FastAPI ? Il fournit validation Pydantic, documentation OpenAPI et tests simples autour d'un service Python.
- La vue 3D représente-t-elle un moteur réel ? Non. Elle explique les signaux et l'état sélectionné, sans prétendre être un modèle physique.

Aya Moujoud

Diapositives 15 à 17. Durée cible : 4 minutes 30.

Texte à présenter

« J'ai rejoint la phase finale avec une responsabilité précise : la validation QA, le monitoring, le contrôle Komodo et la préparation des limites. Mon rôle est de vérifier que les éléments présentés correspondent à des preuves observables, et pas seulement à une configuration écrite dans le dépôt. »

« Pour la validation de production, je contrôle d'abord les services FastAPI, MLflow et Dagster. Je vérifie ensuite les codes HTTP de santé et de prédiction, la version du champion et la latence. Le contrôle daté du 17 septembre renvoie HTTP 200 pour la santé et la prédiction. Le champion chargé est la version 2. La requête FD004_204 renvoie 101,87 cycles, un risque faible et une latence de 224 millisecondes. »

« Dans Komodo, je vérifie la Stack `industrial_equipment_health_platform`, les conteneurs, les logs et l'état des services. Juste avant la démo, la checklist impose de relever le commit réellement déployé, de confirmer que l'image active correspond à ce commit et de refaire les captures. Nous ne créons pas de ressources artificielles pour remplir les pages Komodo. Nous montrons uniquement les ressources réellement utilisées par la Stack. »

« Le monitoring actuel suit la santé du service, la latence, les journaux de prédiction et un indicateur de drift basé sur les RUL. Ce signal reste limité : il ne couvre pas tout le drift des variables ni le concept drift. Il ne déclenche pas le réentraînement. Dagster réentraîne selon le planning quotidien. »

« Les autres limites sont importantes. C-MAPSS contient des données simulées. Les niveaux de risque proviennent de seuils sur la RUL et non d'un classifieur calibré avec des coûts industriels. FD001 est utilisé pour alléger la production de démonstration, alors que l'évaluation complète couvre les quatre sous-ensembles. Les améliorations prioritaires sont des données réelles, un monitoring par variable et une validation métier des alertes. »

« Nous passons maintenant à la démonstration. »

Décision technique à défendre

Une mesure n'est annoncée comme preuve de production que si elle est datée, traçable et reproductible. Les limites sont présentées avec le résultat pour éviter de transformer un indicateur académique en promesse industrielle.

Questions probables

- Aya a-t-elle une contribution dans le dépôt ? Oui. Son périmètre est documenté

dans `team/08-aya-moujoud-qa-monitoring.md`, la fiche de validation, la checklist de démonstration et les scripts de soutenance.

- Le drift déclenche-t-il le modèle ? Non. Il informe le monitoring. Le job quotidien Dagster exécute le réentraînement.

- Pourquoi ne pas remplir toutes les pages Komodo ? Une ressource n'est créée que si elle correspond à l'architecture réelle. La Stack et ses services sont les preuves pertinentes.

- Quelle preuve montrer en premier ? La Stack et les conteneurs sains, puis `/health`, la prédiction, le champion MLflow et le planning Dagster.

7. Démonstration en direct

Un scénario de cinq minutes qui relie le produit aux preuves MLOps.

Diapositive 18. Durée maximale : 5 minutes.

Une seule personne pilote l'écran. Les autres prennent la parole uniquement au moment de leur preuve.

0:00 à 0:45, Hajar

Ouvrir l'application. Montrer la flotte et sélectionner un moteur. Dire :

« L'interface interroge la même API que celle testée dans la CI. Nous allons suivre un moteur, exécuter une prédiction et vérifier les preuves MLOps. »

0:45 à 1:30, Hajar

Afficher l'historique, lancer `POST /predict` et montrer la RUL, le risque, la version et la latence.

1:30 à 2:10, Ossama

Ouvrir MLflow. Montrer le modèle enregistré et l'alias champion sur la version

- Ne pas réciter un ancien run ID. Ouvrir le run visible dans l'interface.

2:10 à 2:50, Mohamed

Ouvrir Dagster. Montrer les assets et le planning quotidien à 06:00. Préciser :

« Le drift est produit après le training. Il ne déclenche pas le job. »

2:50 à 3:40, Aya

Ouvrir Komodo. Montrer la Stack, les conteneurs et un état de santé ou un log récent. Confirmer la révision active selon la checklist.

3:40 à 4:20, Hamza

Ouvrir GitHub Actions et montrer le run réussi. Relier le contrôle de qualité, le build Docker et le smoke test.

4:20 à 4:45, Mouhcine

Montrer le rapport d'évaluation. Rappeler les valeurs finales : MAE 12,56, RMSE 16,93 et score NASA 4 466 sur 707 moteurs.

4:45 à 5:00, Ilyass

Conclure :

« La démonstration relie donc les données, les contrôles, le modèle versionné et le service réellement déployé. Nous sommes prêts pour vos questions. »

Plan de secours

Si un service externe ne répond pas, utiliser les captures datées, les réponses JSON et les rapports conservés dans le dépôt. Ne jamais annoncer une valeur qui ne peut pas être reliée à une preuve.

8. Préparation de la démonstration

Contrôles à effectuer avant de commencer.

La veille

- Confirmer que la Stack Komodo est en état Running et que les conteneurs API, MLflow et Dagster sont sains.
- Vérifier que la version déployée correspond au commit que vous allez annoncer.
- Ouvrir MLflow et confirmer l'alias champion ainsi que la version réellement chargée.
- Ouvrir Dagster et confirmer le planning quotidien à 06:00.
- Ouvrir le dernier workflow GitHub Actions vert et noter son commit.
- Exécuter une prédiction et conserver la réponse JSON ainsi qu'une capture.
- Exporter les preuves hors ligne dans un dossier unique.

Quinze minutes avant

```
git status --short
git rev-parse HEAD
docker compose -f deploy/compose.production.yml config --quiet
docker compose -f deploy/compose.production.yml ps
curl -f http://exp.s3.fsbm.ma:3402/health
curl -f http://exp.s3.fsbm.ma:3402/demo-payload -o /tmp/demo-payload.json
curl -f -H 'Content-Type: application/json' --data @/tmp/demo-payload.json
http://exp.s3.fsbm.ma:3402/predict
```

Onglets à ouvrir

- Application, vue flotte déjà chargée.
- FastAPI documentation ouverte sur POST /predict.
- MLflow ouvert sur le modèle enregistré et son alias champion.
- Dagster ouvert sur les assets et le schedule.
- Komodo ouvert sur la Stack et la liste des conteneurs.
- GitHub Actions ouvert sur le dernier run vert.
- Rapport d'évaluation ouvert sur les métriques finales.

9. Ce qu'il faut montrer dans chaque outil

Le professeur doit voir une preuve, pas seulement entendre le nom de l'outil.

Outil	Preuve à afficher	Phrase courte
Application	Flotte, moteur, historique et résultat	La décision visible vient de la même API que celle testée.
FastAPI	GET /health, POST /predict et réponse	Le contrat valide l'entrée et renvoie RUL, risque, modèle et latence.
MLflow	Run, métriques, artefacts, modèle et alias	L'API charge l'alias champion et non un numéro codé.
Dagster	Assets, dépendances, dernier run et schedule	Le job complet est planifié chaque jour à 06:00.
Komodo	Stack, conteneurs, état, logs et serveur	Komodo gère le cycle de vie de la Stack sur le serveur partagé.
GitHub	Code, branches, PR et documentation	Le dépôt conserve le code, les responsabilités et les preuves.
Actions	Run vert, tests, build et smoke	La livraison est bloquée si la qualité ou le démarrage échoue.
Rapports	MAE, RMSE, NASA et drift	Les chiffres annoncés sont reliés à des artefacts versionnés.

10. Questions probables du professeur

Réponses courtes, exactes et défendables.

Quel est l'objectif métier ?

Estimer la durée de vie restante d'un moteur pour prioriser une maintenance avant la panne.

Pourquoi NASA C-MAPSS ?

Il fournit des trajectoires multivariées jusqu'à la défaillance avec une vérité terrain RUL et un protocole reproductible.

Pourquoi plafonner la RUL à 125 cycles ?

La longue phase initiale contient peu de signal de dégradation. Le plafonnement réduit son influence sur l'apprentissage.

Comment évitez-vous la fuite de données ?

La séparation est faite par moteur et respecte l'ordre temporel. Un moteur de validation n'apparaît jamais dans l'entraînement.

Pourquoi des fenêtres de cinq cycles ?

Elles capturent la tendance récente avec moyenne, écart type et pente sans trop lisser les changements.

Pourquoi HistGradientBoosting ?

Il donne le meilleur compromis sur MAE, RMSE et score NASA avec une inférence adaptée au service.

Pourquoi le score NASA ?

Il pénalise davantage la surestimation de la RUL, plus risquée dans un contexte de maintenance.

Que fait MLflow ?

Il trace les paramètres, métriques et artefacts, enregistre le modèle et gère l'alias champion.

Comment un modèle devient champion ?

Le candidat est promu seulement si sa MAE et sa RMSE ne se dégradent pas par rapport au champion.

Le réentraînement est-il automatique ?

Oui. Dagster lance le pipeline chaque jour à 06:00. Ce déclenchement est planifié.

Le drift déclenche-t-il un entraînement ?

Non. Le drift informe le monitoring. Il ne déclenche pas le job.

Pourquoi dlt et dbt ?

dlt charge les données et gère le schéma. dbt transforme les tables, teste les règles et documente le lignage SQL.

Pourquoi DuckDB ?

Le volume tient sur une machine. DuckDB apporte SQL et reproductibilité sans administration d'un serveur de base de données.

Que garantit la CI ?

Elle exécute les contrats, tests, entraînement, build et smoke test sur un commit précis. Elle ne garantit pas les futures données réelles.

Pourquoi Docker ?

Docker fixe les dépendances et rend le même service reproductible en CI et sur le serveur.

Pourquoi Komodo ?

Komodo gère la Stack, les conteneurs, les logs, les mises à jour et l'observation sur le serveur partagé.

Pourquoi FastAPI ?

FastAPI fournit un contrat Pydantic strict, OpenAPI, des tests simples et un service Python adapté au modèle.

Quelles sont les limites ?

Les données sont simulées, le drift est partiel, les seuils de risque ne sont pas calibrés sur un coût industriel et FD001 allège la production de démonstration.

Que feriez-vous ensuite ?

Connecter une vraie télémétrie, suivre le drift par variable, valider les seuils avec le métier et ajouter une règle de réentraînement fondée sur des critères approuvés.

Que fait chaque membre ?

Chaque rôle correspond à une partie vérifiable du pipeline. La répartition et les livrables sont documentés dans le README, CONTRIBUTORS et les fiches d'équipe.

11. Aya dans GitHub

Pourquoi elle n'apparaît pas encore dans les deux zones montrées sur les captures.

Le bloc Team du README

La branche de la pull request 12 contient déjà Aya dans le tableau Team et retire Akram de la liste des présentateurs actifs. La capture montre encore la branche principale. Tant que la pull request n'est pas approuvée et fusionnée, GitHub affiche l'ancien README sur la page principale.

Le graphe Contributors

Le graphe Contributors n'est pas la liste des collaborateurs. Il est calculé à partir des commits attribués sur la branche par défaut. Aya possède déjà l'accès write, mais cet accès ne crée pas un commit et ne modifie donc pas ce graphe.

Procédure correcte

Aya doit effectuer un vrai changement depuis son propre compte, avec une adresse vérifiée, puis ouvrir une pull request. Après revue et fusion dans main, GitHub pourra lui attribuer ce commit et afficher son profil dans Contributors. Il ne faut pas fabriquer un commit en son nom.

Contribution adaptée à Aya

- Exécuter la checklist finale de production.
- Compléter la fiche de validation avec les résultats et captures datées.
- Vérifier Komodo, FastAPI, MLflow, Dagster et le monitoring.
- Documenter les limites constatées sans modifier les résultats scientifiques.
- Soumettre cette preuve depuis son compte ayamoujoud.

12. Plan de secours

À utiliser si le réseau ou un service externe tombe pendant la soutenance.

- Capture de la Stack Komodo et des conteneurs sains.
- Capture de MLflow montrant le modèle et l'alias champion.
- Capture de Dagster montrant les assets et le planning.
- Réponses JSON de /health, /predict, /api/model/info et /api/monitoring/summary.
- Rapport final_evaluation.json avec MAE, RMSE et score NASA.
- Rapport promotion_decision.json.
- Rapport drift_report.json avec sa date et sa source.
- Capture du dernier GitHub Actions vert.

Règle

Ne jamais annoncer une valeur qui ne peut pas être reliée à une capture, une réponse JSON, un rapport ou un écran de la plateforme.

13. Conclusion à retenir

La phrase qui résume le projet en vingt secondes.

« Nous avons construit une chaîne MLOps et DataOps complète pour estimer la durée de vie restante d'un moteur. Les données NASA sont ingérées et contrôlées, les modèles sont comparés et suivis dans MLflow, le champion est servi par FastAPI, puis Docker et Komodo rendent la solution reproductible et observable. La démonstration relie chaque résultat à une preuve technique. »

Fin du guide. Refaire la checklist et noter les valeurs dynamiques le jour de la soutenance.